CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 8

Database Design for StudentHub – Students, Events and Registrations in the Shared Portal Database

MariaDB 10.4 (studenthub_db) · migration 001 · InnoDB keys and constraints · PHP 8 MySQLi · phpMyAdmin

Corrected submission – implemented in the integrated StudentHub portal (studenthub/)

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

Corrected report. This report replaces the original Practical 8 report (`Practical-8/PRAC_8_ITUE203_25CS023.docx`, kept unchanged in the repository). The original report describes a standalone database `studenthub_prac8` (UNSIGNED keys, two stored procedures, a PDO test page `test_connection.php`). This corrected version describes the same design as it now exists in the shared portal database `studenthub_db`: migration 001 adds the `registrations` table, the student–account link, event time and capacity and the CHECK and foreign-key rules to the existing portal tables without replacing them; the capacity rule is a PHP transaction instead of a stored procedure, and the portal connects with MySQLi. All screenshots in this report were taken from the canonical integrated application `studenthub/` with Microsoft Edge on disposable test databases with fictional records; all test results are copied from the saved test outputs of the repository.

1. Aim and Objectives

To design a normalised MySQL/MariaDB database for StudentHub with students, events and registrations and to connect to it securely from PHP with prepared statements – as the database of the one integrated portal (`studenthub/`), so that registration, login, event registration and the Admin Console all use the same records.

- Model the data with an entity–relationship view and keep the tables in Third Normal Form (3NF).
- Define primary, foreign and unique keys, CHECK constraints, indexes and suitable data types.
- Add the design to the existing portal database without changing any existing value (migration with preconditions, verification and rollback), and provide fictional demo data for new installations (no accounts, no password hashes).
- Connect from PHP without ever showing credentials or database errors, use prepared statements for every query with a value, and enforce the event capacity without overbooking, also for simultaneous requests.
- Test every part on disposable databases and record only results that were actually executed.

2. Problem Definition

Official problem definition (from the original report): *"Design the database for StudentHub with students, events and registrations. Create PHP database connection using MYSQL statements."*

A student registers for many events and an event has many students, so the relationship must be resolved without repeating student or event details. The database must refuse invalid data – a registration for a missing student or event, the same student twice for one event, a semester outside 1–8, a capacity of zero – and PHP must connect safely. In the portal there was a second problem: `studenthub_db` already existed with its own column names (`enrollment_no`, `name`, `course`, `location`), signed `INT(11)` keys, the roles admin, editor and student and real accounts. A separate Practical 8 database would have created a second, diverging set of students and events, so the design had to be **added** to the existing tables by a migration that changes no existing value.

Key question	Answer in the integrated portal
Is the schema normalised properly?	Yes – student data only in `students`, event data only in `events`, the many-to-many relationship resolved by `registrations`; single surrogate keys, no transitive dependencies (Section 5.3).
Are primary and foreign keys correctly defined?	Every table has an `AUTO_INCREMENT` primary key; `registrations` has two CASCADE foreign keys and a unique (student_id, event_id) pair; the tests confirmed that invalid references are refused (Section 10).
Is the connection handled securely?	Yes – MySQLi in exception mode, utf8mb4, settings from the environment, no fallback database, a general HTTP 503 page and the real reason only in the server log (Section 7).

3. Requirements and Tools

Item	Version / use
XAMPP (Windows 11)	PHP 8.2.12 with `mysqli`; MariaDB 10.4.32 (database `studenthub_db`, InnoDB, `utf8mb4_unicode_ci`); phpMyAdmin
mysql command-line client	`C:\xampp\mysql\bin\mysql.exe` – applies the migration and the demo file (`database/README.md`)
phpMyAdmin	XAMPP's phpMyAdmin (`http://localhost/phpmyadmin`) – tables, relations, Designer view and a JOIN query (Figures 1–6)
Python 3 / Microsoft Edge + Playwright	`database/tests/test_001_practical8_foundation.py`, `tests/run_all.py`; `tests/browser_database_evidence.py` (screenshots)
Git / GitHub	Commit `37b35e6` – Practical 8 foundation in the portal database

4. Place in the Integrated Portal

Practical 8 is the **database foundation** of the portal. The files below define, document and test `studenthub_db`; the portal's PHP code uses it through one connection.

File	Role
`database/migrations/001_practical8_foundation.sql` (+ `.down.sql`)	Upgrades an existing `studenthub_db` to the Practical 8 design; schema rollback
`database/studenthub_demo.sql`	Current schema (migrations 001–005) with fictional demo rows for a new installation
`database/README.md`	Relationships, Practical 8 mapping, setup, rollback, MariaDB DDL limitations, test results
`studenthub/db.php`	The portal's MySQLi connection
`studenthub/includes/event_registration.php`	`register_student_for_event()` – capacity rule in one transaction (replaces the stored procedure)
`database/tests/test_001_practical8_foundation.py`	Migration and behaviour tests on disposable databases plus a read-only live check

`database/README.md` maps the original design to the portal database:

Practical 8 (studenthub_prac8)	Portal database (studenthub_db)	Note
`registrations` (CASCADE FKs, UNIQUE student+event, status ENUM)	`registrations` – same columns and rules	keys signed `INT(11)` like the portal's primary keys (P8: UNSIGNED)
`students.semester` TINYINT, CHECK 1–8	`INT(11)`, CHECK NULL or 1–8	NULL allowed for legacy rows
`events.capacity` NOT NULL, CHECK > 0	NULL, CHECK NULL or > 0; new `event_time`	existing events: NULL = no limit (nothing invented)
`student_code`, `full_name`, `venue`, `branch`	`enrollment_no`, `name`, `location`, `course`	portal names kept; course names stay free text
procedure `register_student_for_event`	PHP function of the same name	the same rule inside a real transaction
–	`students.user_id` → `users.id`; `audit_logs.user_id` → `users.id`	student–account link; optional audit link

5. Database Design

5.1 Tables and relationships

After migration 001 the Practical 8 part of `studenthub_db` consists of these tables (InnoDB, `utf8mb4_unicode_ci`, signed `INT(11)` keys). Columns added by later migrations are marked in italics.

Table	Columns	Purpose
`students`	`id` PK, `user_id` (FK, UNIQUE, NULL), `enrollment_no`, `name`, `email` (UNIQUE), `course`, `gender`, `semester`, `status`, `created_at`	One row per student (also without an account)
`events`	`id` PK, `title`, `event_date`, `event_time`, `location`, `capacity`, `description`, `category`, `poster`, `created_at` – *`status, updated_at (003)`*	One row per event
`registrations`	`registration_id` PK, `student_id` FK, `event_id` FK, `registered_at`, `status` ENUM('registered','attended','cancelled')	One row per student per event
`users`, `audit_logs`	login accounts (Practicals 9/10) and the activity log, now linked by foreign keys	Accounts and history
`schema_migrations`	`version` PK, `description`, `applied_at`	Applied migrations

Relationship	Foreign key	Cardinality	On delete / update
students → registrations	`fk_registrations_student`: `student_id` → `students.id`	1 : n	CASCADE / CASCADE
events → registrations	`fk_registrations_event`: `event_id` → `events.id`	1 : n	CASCADE / CASCADE
users → students	`fk_students_user`: `students.user_id` → `users.id` (UNIQUE)	1 : 0..1	SET NULL / CASCADE
users → audit_logs	`fk_audit_logs_user`: `audit_logs.user_id` → `users.id`	1 : n	SET NULL / CASCADE

Instead of the hand-drawn ER diagram of the original, the ER view of the portal database was generated by phpMyAdmin's Designer from the real foreign keys (Figure 5); its automatic layout lets some boxes overlap, so the table above is the authoritative list of relationships.

5.2 Normalisation to Third Normal Form

The three-table JOIN of Figure 6 returns one row per registration. Stored as a single table, every row would repeat the event's title, date and location (values from `studenthub_demo.sql` and the fictional registrations of the screenshot):

enrollment_no	name	event title	event_date	location	status
25CE045	Diya Shah	Full-Stack Web Dev Bootcamp	2026-11-20	CSPIT Lab 3	registered
25IT012	Meet Desai	Full-Stack Web Dev Bootcamp	2026-11-20	CSPIT Lab 3	registered
25CE078	Pooja Sharma	Odoo Hackathon 2026	2026-08-22	LDCE Ahmedabad	attended

That design has update, insert and delete anomalies (a new location must be changed in every row; an event without registrations cannot be stored; cancelling the last registration loses the student). The portal schema avoids them:

Normal	How the portal schema meets it

form	
1NF	Every column holds one atomic value, there are no repeating groups and every table has a primary key.
2NF	Every primary key is a single surrogate column, so no column depends on part of a key; the natural pair (student_id, event_id) is a UNIQUE key and `registered_at` / `status` depend on the whole registration.
3NF	No non-key column depends on another non-key column: student details depend only on `students.id`, event details only on `events.id`; `registrations` holds only the two foreign keys and facts of the registration.

`students.email`, `students.user_id` and (migration 002) `students.enrollment_no` are unique, so they are candidate keys and the tables stay in 3NF. Two redundancies remain and are listed in Section 12: course names are free text, and a self-registered student's e-mail is stored in `users` and in `students`.

5.3 Keys, constraints and indexes

Element	Definition in studenthub_db
Primary keys	`INT(11) AUTO_INCREMENT` – signed, because a foreign-key column must have exactly the referenced type and the existing portal keys were signed.
Foreign keys	registrations → students / events with ON DELETE CASCADE; `students.user_id` and `audit_logs.user_id` → `users.id` with ON DELETE SET NULL (deleting an account keeps the student record and the audit history).
Unique keys	`uq_registrations_student_event` (student_id, event_id); `uq_students_user_id`; `students.email`; `users.username`, `users.email`; later `uq_students_enrollment_no` (002).
CHECK	`chk_students_semester`: NULL or 1–8; `chk_events_capacity`: NULL or > 0 (enforced by MariaDB independently of `sql_mode`).
Indexes	`idx_registrations_event` (event_id); `idx_events_date` (event_date, event_time); the FK `student_id` uses the leftmost column of the unique key (Figure 2).

5.4 Migration 001

Step 1 of the migration runs only read-only checks; if one fails, the script stops with a message and nothing is changed:

database/migrations/001_practical8_foundation.sql (lines 29–56) – preconditions (nothing changes if one fails)

```
29  -- 0. Preconditions (read-only)
30  -- ------------------------------------------------------------------------------------------------
    -----------------
31  BEGIN NOT ATOMIC
32      IF DATABASE() IS NULL THEN
33          SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Migration 001: no database selected. Run: mysql -u
    root studenthub_db < 001_practical8_foundation.sql';
34      END IF;
35      IF (SELECT COUNT(*) FROM information_schema.TABLES
36          WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME IN ('users', 'students', 'events', 'audit_logs')
    AND ENGINE = 'InnoDB') <> 4 THEN
37          SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Migration 001: tables users, students, events and
    audit_logs (InnoDB) are required.';
38      END IF;
39      IF (SELECT COUNT(*) FROM information_schema.COLUMNS
40          WHERE TABLE_SCHEMA = DATABASE() AND COLUMN_TYPE = 'int(11)'
41            AND ((TABLE_NAME = 'users' AND COLUMN_NAME = 'id') OR (TABLE_NAME = 'students' AND
    COLUMN_NAME = 'id')
42              OR (TABLE_NAME = 'events' AND COLUMN_NAME = 'id') OR (TABLE_NAME = 'audit_logs' AND
    COLUMN_NAME = 'user_id'))) <> 4 THEN
43          SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Migration 001: users.id, students.id, events.id and
    audit_logs.user_id must be signed INT(11).';
44      END IF;
45      IF EXISTS (SELECT 1 FROM audit_logs a LEFT JOIN users u ON u.id = a.user_id WHERE a.user_id IS NOT
    NULL AND u.id IS NULL) THEN
```

```
46          SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Migration 001: audit_logs has user_id values of
    deleted accounts. Review them (see database/README.md) before migrating.';
47      END IF;
48      IF EXISTS (SELECT 1 FROM students WHERE semester IS NOT NULL AND semester NOT BETWEEN 1 AND 8) THEN
49          SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Migration 001: students has semester values outside
    1-8. Correct them before migrating.';
50      END IF;
51      IF EXISTS (SELECT 1 FROM information_schema.TABLES WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME =
    'registrations')
52         AND NOT EXISTS (SELECT 1 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA = DATABASE() AND
    TABLE_NAME = 'registrations'
53                         AND COLUMN_NAME = 'registration_id' AND COLUMN_TYPE = 'int(11)') THEN
54          SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Migration 001: a different registrations table
    already exists. Rename or remove it first.';
55      END IF;
56  END //
```

Next, `students` receives the account link and Practical 8's semester rule. The **linking rule** is strict: a record gets a `user_id` only if its e-mail (trimmed, case-insensitive) belongs to exactly one account, that account has the role `student`, exactly one student record has that e-mail and neither side is linked yet. Everything else stays unlinked; no account is created and nothing is deleted.

database/migrations/001_practical8_foundation.sql (lines 61–87) – students.user_id, the semester CHECK and the verified e-mail linking

```
61  -- 1. students: link to the login account and Practical 8's semester rule
62  -- ------------------------------------------------------------------------------------------------------
    -----------------
63  ALTER TABLE students
64      ADD COLUMN IF NOT EXISTS user_id INT(11) NULL DEFAULT NULL AFTER id,
65      ADD UNIQUE KEY IF NOT EXISTS uq_students_user_id (user_id);
66
67  ALTER TABLE students
68      ADD CONSTRAINT fk_students_user FOREIGN KEY IF NOT EXISTS (user_id) REFERENCES users (id)
69          ON DELETE SET NULL ON UPDATE CASCADE;
70
71  ALTER TABLE students
72      ADD CONSTRAINT IF NOT EXISTS chk_students_semester CHECK (semester IS NULL OR semester BETWEEN 1
    AND 8);
73
74  -- Link a student record to an account only when the e-mail (trimmed, case-insensitive) belongs to
    exactly one account,
75  -- that account has the role 'student', exactly one student record has that e-mail, and neither side is
    linked yet.
76  -- Everything else (no account, ambiguous e-mail, admin/editor account) stays unlinked; no account is
    created.
77  UPDATE students AS s
78  JOIN (
79      SELECT MIN(st.id) AS student_id, MIN(u.id) AS user_id
80      FROM users AS u
81      JOIN students AS st ON LOWER(TRIM(st.email)) = LOWER(TRIM(u.email))
82      GROUP BY LOWER(TRIM(u.email))
83      HAVING COUNT(DISTINCT st.id) = 1 AND COUNT(DISTINCT u.id) = 1 AND MIN(u.role) = 'student'
84  ) AS verified ON verified.student_id = s.id
85  LEFT JOIN students AS already ON already.user_id = verified.user_id
86  SET s.user_id = verified.user_id
87  WHERE s.user_id IS NULL AND already.id IS NULL;
```

Then event time and capacity are added (NULL for every existing event), the `registrations` table is created with both CASCADE foreign keys, and `audit_logs` gets its optional foreign key:

database/migrations/001_practical8_foundation.sql (lines 90–122) – events.event_time / capacity, registrations and fk_audit_logs_user

```
90  -- 2. events: time and capacity (NULL for existing events - nothing is invented)
91  -- ------------------------------------------------------------------------------------------------------
    -----------------
92  ALTER TABLE events
93      ADD COLUMN IF NOT EXISTS event_time TIME NULL DEFAULT NULL AFTER event_date,
94      ADD COLUMN IF NOT EXISTS capacity SMALLINT(5) UNSIGNED NULL DEFAULT NULL AFTER location,
95      ADD INDEX IF NOT EXISTS idx_events_date (event_date, event_time);
96
```

```
 97  ALTER TABLE events
 98      ADD CONSTRAINT IF NOT EXISTS chk_events_capacity CHECK (capacity IS NULL OR capacity > 0);
 99
100  -- ------------------------------------------------------------------------------------------------------------
     -----------------
101  -- 3. registrations: Practical 8's many-to-many table between students and events
102  --    (capacity is enforced in PHP with a transaction that locks the event row:
     studenthub/includes/event_registration.php)
103  -- ------------------------------------------------------------------------------------------------------------
     -----------------
104  CREATE TABLE IF NOT EXISTS registrations (
105      registration_id INT(11)  NOT NULL AUTO_INCREMENT,
106      student_id      INT(11)  NOT NULL,
107      event_id        INT(11)  NOT NULL,
108      registered_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
109      status          ENUM('registered', 'attended', 'cancelled') NOT NULL DEFAULT 'registered',
110      PRIMARY KEY (registration_id),
111      UNIQUE KEY uq_registrations_student_event (student_id, event_id),
112      KEY idx_registrations_event (event_id),
113      CONSTRAINT fk_registrations_student FOREIGN KEY (student_id) REFERENCES students (id) ON DELETE
     CASCADE ON UPDATE CASCADE,
114      CONSTRAINT fk_registrations_event FOREIGN KEY (event_id) REFERENCES events (id) ON DELETE CASCADE
     ON UPDATE CASCADE
115  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
116
117  -- ------------------------------------------------------------------------------------------------------------
     -----------------
118  -- 4. audit_logs: optional link to the account; anonymous entries keep user_id NULL, deleted accounts
     become NULL
119  -- ------------------------------------------------------------------------------------------------------------
     -----------------
120  ALTER TABLE audit_logs
121      ADD CONSTRAINT fk_audit_logs_user FOREIGN KEY IF NOT EXISTS (user_id) REFERENCES users (id)
122          ON DELETE SET NULL ON UPDATE CASCADE;
```

A final block verifies every foreign key (with its referenced table and delete rule, because `ADD … IF NOT EXISTS` compares only names), both CHECK constraints and both unique keys, and only then records version `001` in `schema_migrations`. Every step is idempotent. On the live database the migration linked 5 student records; 4 records have no account and 1 student account has no record (`database/README.md`).

Rollback and DDL limitations. The preferred way back is the `mysqldump` taken before migrating; `001_practical8_foundation.down.sql` removes exactly what 001 added (and therefore deletes all registrations, event times/capacities and links). MariaDB commits every `ALTER/CREATE TABLE` immediately, so the migration is not atomic: the preconditions catch known causes first and a re-run completes an interrupted migration. XAMPP runs without strict SQL mode, so values not covered by a CHECK (for example too-long text) are still silently adjusted.

5.5 Demo data

`database/studenthub_demo.sql` creates the current schema (with migrations 001–005), refuses to run on a database older than 001 and can be re-run safely. It contains 7 students (the owner's record and 6 fictional `example.com` students, one Inactive), 5 events (capacity and time NULL), 7 event categories, 9 menu links and the 5 migration rows – but no accounts, no password hashes, no audit entries and no registrations. For the screenshots, `tests/browser_database_evidence.py` loaded it into the disposable database `studenthub_report_demo`, set event 5 to 2026-11-20 with capacity 40 and inserted four fictional registrations; the database was dropped afterwards.

6. Setting Up the Database

A **new installation** creates the database and imports the demo file with the mysql client (`database/README.md`):

1. `mysql.exe -u root -e "CREATE DATABASE IF NOT EXISTS studenthub_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci"`

2. `mysql.exe -u root studenthub_db < database\studenthub_demo.sql`, then create the first administrator with `tools/create_demo_user.php` (the password is prompted, never stored).

An **existing database** is backed up with `mysqldump` (kept outside the repository – it contains real accounts and hashes), migrated with `mysql.exe -u root studenthub_db < database\migrations\001_practical8_foundation.sql` and checked with `test_001_practical8_foundation.py --verify-live --baseline-dump ….` The original practical imported its `schema.sql` through phpMyAdmin's Import tab; here phpMyAdmin is used to inspect the result (Figures 1–6). The disposable database for these screenshots was loaded with the mysql client; a phpMyAdmin import was not captured again.

7. Secure Database Connection (studenthub/db.php)

Every page includes `studenthub/db.php`, which opens one MySQLi connection. Settings come from the environment with XAMPP defaults: `DB_HOST` (localhost), `DB_USER` (root), `DB_PASS` (empty), `DB_NAME` (`studenthub_db`). `display_errors` is off and `log_errors` on.

studenthub/db.php (lines 53–70) – exception mode, utf8mb4, no fallback, general error page

```
53  // Uncaught errors: log the details, show a general page
54  set_exception_handler(function (Throwable $e): void {
55      error_log('StudentHub error: ' . get_class($e) . ' [' . $e->getCode() . '] ' . $e->getMessage() .
    ' in ' . $e->getFile() . ':' . $e->getLine());
56      studenthub_fail(500, 'Something went wrong', 'The page could not be completed. Please try again
    later.');
57  });
58
59  mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);   // every MySQL error throws
    mysqli_sql_exception
60  try {
61      $conn = new mysqli($db_host, $db_user, $db_pass, $db_name);
62      $conn->set_charset('utf8mb4');
63  } catch (mysqli_sql_exception $e) {
64      // Diagnostic for the administrator (server log only); the password is never logged
65      error_log('StudentHub: cannot connect to the configured database "' . $db_name . '" on host "' .
    $db_host . '" as user "'
66          . $db_user . '": [' . $e->getCode() . '] ' . $e->getMessage() . '. No fallback database is
    used.');
67      studenthub_fail(503, 'Service temporarily unavailable', 'StudentHub cannot reach its database
    right now. Please try again in a few minutes.');
68      exit;
69  }
70  unset($db_pass);
```

- **No fallback:** if the configured database is unavailable the portal answers HTTP 503 – it never falls back to an older database such as `user_registration`. `tests/test_auth.py` (132 / 132) checked this with an unknown database and a wrong password: HTTP 503 on the home, sign-in and dashboard pages, no credentials, database names, SQL or stack traces in the page, the real reason only in the server log and the password never logged.
- **Exception mode:** `mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT)` makes every MySQL error throw; uncaught exceptions reach `set_exception_handler()`, are logged and answered with a general HTTP 500 page.
- **Nothing leaks:** `studenthub_fail()` prints only a general message (JSON for the API) with `Cache-Control: no-store`; the log line names database, host and user but never the password, and `unset($db_pass)` removes it after connecting.
- **UTF-8 and time zone:** `set_charset('utf8mb4')`; dates use `Asia/Kolkata` like MariaDB on this server.

`db.php` also opens a PDO handle (`$pdo`) "for later Fetch/AJAX modules"; no page of the portal uses it at present – all queries use the MySQLi connection `$conn`.

8. Prepared Statements and the Capacity Rule

A prepared statement sends the SQL with `?` placeholders first and the values separately (`bind_param()`), so a value can never change the query; the portal's queries with request values are MySQLi prepared statements. The original stored procedure became the PHP function `register_student_for_event()`. It runs in **one transaction**, locks the event row with `SELECT … FOR UPDATE` so that concurrent registrations for the same event wait for each other, and counts the seats taken (registrations that are not `cancelled`) only after the lock is held. `capacity` NULL means no limit.

studenthub/includes/event_registration.php (lines 45–88) – locking, checks and the capacity count

```
45  function register_student_for_event(mysqli $db, int $student_id, int $event_id): int
46  {
47      $db->begin_transaction();
48      try {
49          $stmt = $db->prepare('SELECT capacity, status, event_date < CURDATE() AS is_past FROM events
    WHERE id = ? FOR UPDATE');
50          $stmt->bind_param('i', $event_id);
51          $stmt->execute();
52          $event = $stmt->get_result()->fetch_assoc();
53          if ($event === null) {
54              throw new EventRegistrationException(EventRegistrationException::EVENT_NOT_FOUND);
55          }
56          if ($event['status'] !== 'open') {
57              throw new EventRegistrationException(EventRegistrationException::EVENT_CLOSED);
58          }
59          if ((int) $event['is_past'] === 1) {
60              throw new EventRegistrationException(EventRegistrationException::EVENT_PAST);
61          }
62
63          $stmt = $db->prepare('SELECT status FROM students WHERE id = ? LOCK IN SHARE MODE');
64          $stmt->bind_param('i', $student_id);
65          $stmt->execute();
66          $student = $stmt->get_result()->fetch_assoc();
67          if ($student === null) {
68              throw new EventRegistrationException(EventRegistrationException::STUDENT_NOT_FOUND);
69          }
70          if (strcasecmp(trim((string) $student['status']), 'Inactive') === 0) {
71              throw new EventRegistrationException(EventRegistrationException::STUDENT_INACTIVE);
72          }
73
74          $stmt = $db->prepare('SELECT registration_id, status FROM registrations WHERE student_id = ?
    AND event_id = ? FOR UPDATE');
75          $stmt->bind_param('ii', $student_id, $event_id);
76          $stmt->execute();
77          $existing = $stmt->get_result()->fetch_assoc();
78          if ($existing !== null && $existing['status'] !== 'cancelled') {
79              throw new EventRegistrationException(EventRegistrationException::DUPLICATE);
80          }
81
82          if ($event['capacity'] !== null) {
83              // locking read: always counts the latest committed rows
84              $stmt = $db->prepare("SELECT COUNT(*) FROM registrations WHERE event_id = ? AND status <>
    'cancelled' LOCK IN SHARE MODE");
85              $stmt->bind_param('i', $event_id);
86              $stmt->execute();
87              if ((int) $stmt->get_result()->fetch_row()[0] >= (int) $event['capacity']) {
88                  throw new EventRegistrationException(EventRegistrationException::EVENT_FULL);
```

If a seat is free, the registration is inserted – or the student's cancelled registration is reactivated, so the unique key is never violated – and the transaction commits; a duplicate inserted meanwhile by another request is caught as MySQL error 1062 and reported as `duplicate`. `database/README.md` lists the reasons `event_not_found`, `student_not_found`, `student_inactive`, `duplicate` and `event_full`; the event module (Practical 12) added `event_closed` and `event_past`. A plain `INSERT INTO registrations` would bypass the rule, so the portal always uses this function – from the Campus Events page (`notifications.php`) and the JSON API (`api/events.php`). Reading the three tables together is a prepared JOIN, for example on the admin dashboard (Practical 14):

studenthub/includes/dashboard.php (lines 25–33) – three-table JOIN with a bound LIMIT

```
25  // The newest event registrations with student and event
26  function latest_registrations(mysqli $db, int $limit = 5): array
27  {
28      $stmt = $db->prepare('SELECT r.registered_at, r.status, s.name, s.enrollment_no, e.title,
    e.event_date
29                            FROM registrations r JOIN students s ON s.id = r.student_id JOIN events e
    ON e.id = r.event_id
30                            ORDER BY r.registered_at DESC, r.registration_id DESC LIMIT ?');
31      $stmt->bind_param('i', $limit);
32      $stmt->execute();
33      return $stmt->get_result()->fetch_all(MYSQLI_ASSOC);
```

9. Output

Figures 1–6 are genuine full-page captures from Microsoft Edge (headless, Playwright, window 1366 × 900) of XAMPP's phpMyAdmin, taken by `tests/browser_database_evidence.py` on the disposable database `studenthub_report_demo` (built from `database/studenthub_demo.sql` plus four fictional registrations). The table `zz_report_demo_marker` in these figures is **only the safety marker of the disposable database** – the script drops a database only if it contains this table; it is not part of the portal schema. The left panel also lists other local databases, among them the historical practical databases. Figures 7 and 8 show the `registrations` table in use by later integration steps.

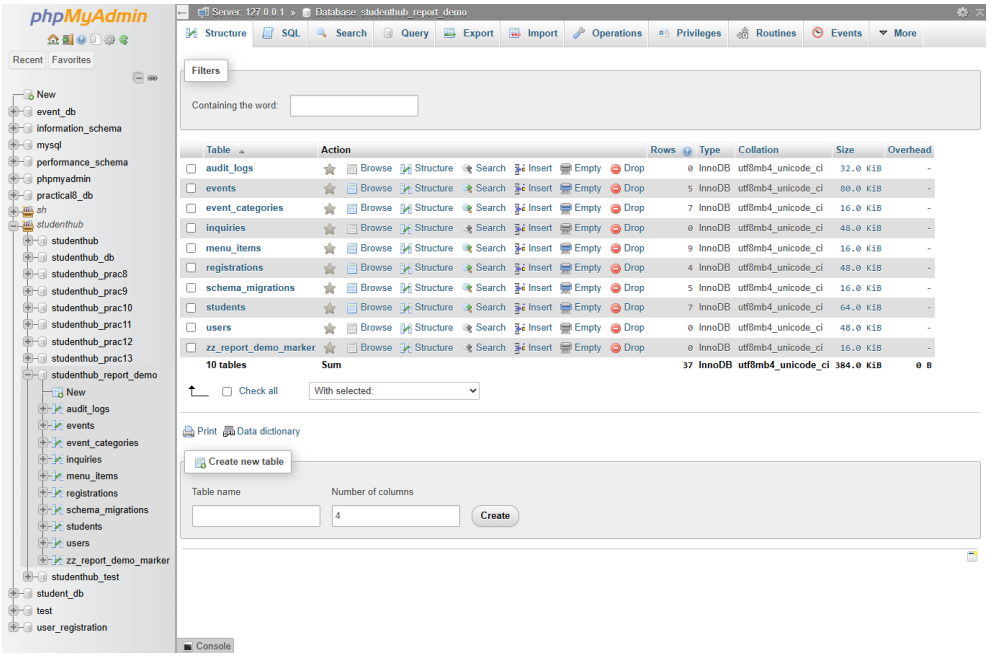


Figure 1: phpMyAdmin – the disposable database studenthub_report_demo: 10 InnoDB tables with utf8mb4_unicode_ci (events 5, event_categories 7, menu_items 9, registrations 4, schema_migrations 5, students 7, users 0, audit_logs 0, inquiries 0 rows) plus the safety marker zz_report_demo_marker (01-database-tables.png)

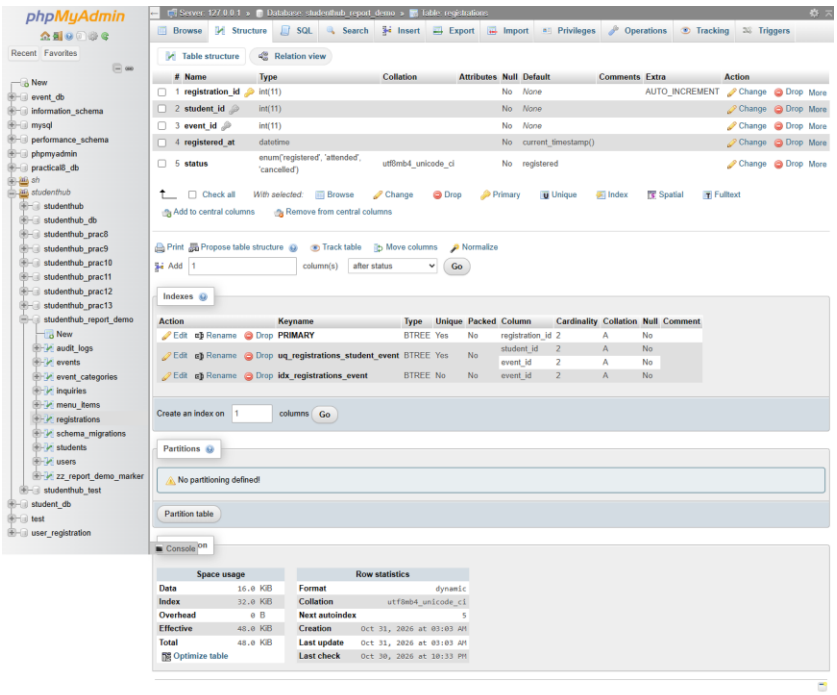


Figure 2: Structure of registrations: registration_id int(11) AUTO_INCREMENT, student_id and event_id int(11), registered_at datetime DEFAULT current_timestamp(), status enum('registered','attended','cancelled'); indexes PRIMARY, uq_registrations_student_event (student_id, event_id) and idx_registrations_event (event_id) (02-registrations-structure.png)

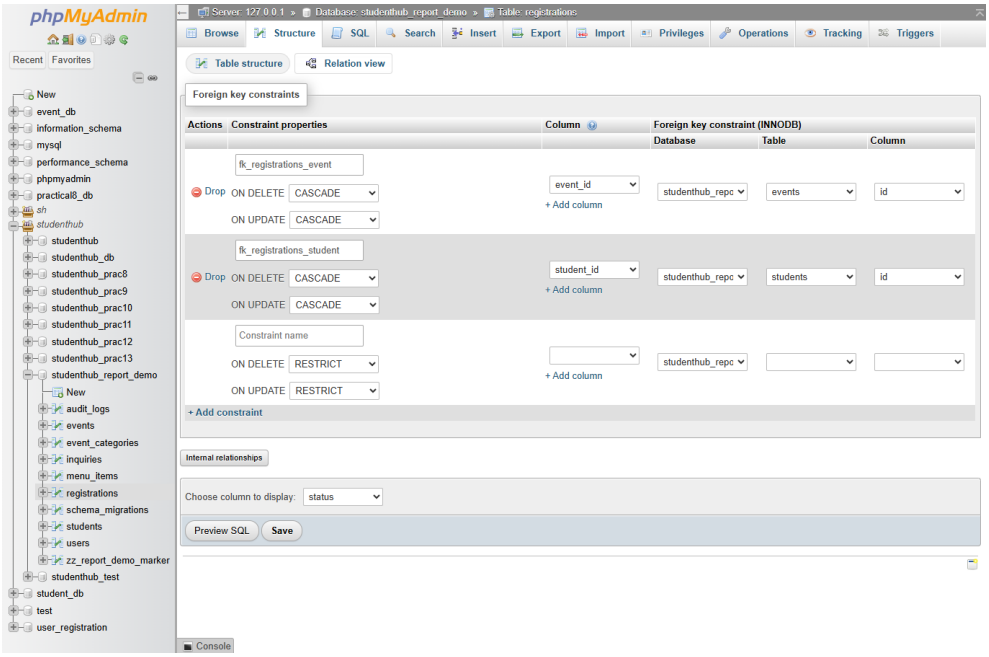


Figure 3: Relation view of registrations: fk_registrations_event (event_id → events.id) and fk_registrations_student (student_id → students.id), both ON DELETE CASCADE and ON UPDATE CASCADE (03-registrations-foreign-keys.png)

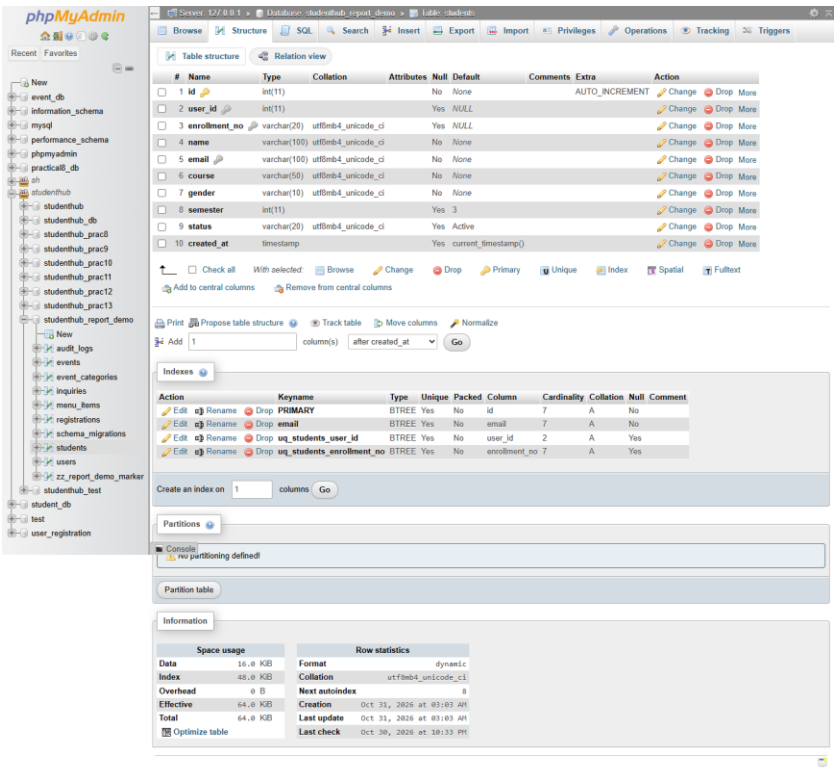


Figure 4: Structure of students after migrations 001 and 002: user_id int(11) NULL, enrollment_no varchar(20) NULL, semester int(11) DEFAULT 3, status DEFAULT Active; unique indexes PRIMARY, email, uq_students_user_id, uq_students_enrollment_no (04-students-structure.png)

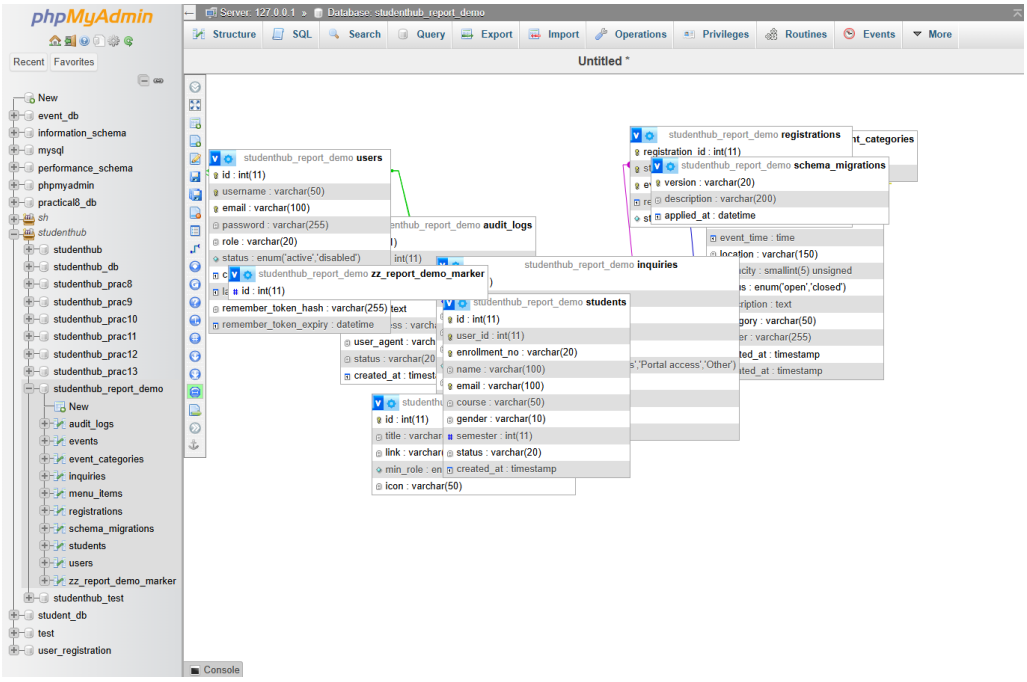


Figure 5: ER view generated by phpMyAdmin's Designer from the real foreign keys; the automatic layout lets some boxes overlap (Section 5.1 lists the relationships). The box zz_report_demo_marker is only the safety marker of the disposable database (05-er-diagram-designer.png)

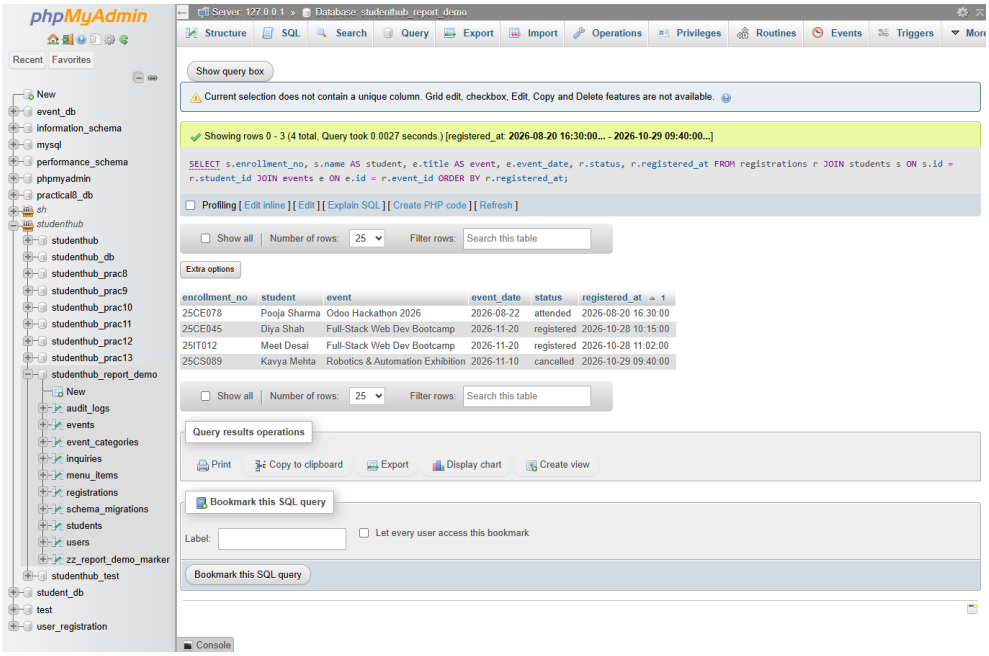


Figure 6: Three-table JOIN of registrations, students and events: the 4 fictional registrations – Pooja Sharma (25CE078) Odoo Hackathon 2026 attended; Diya Shah (25CE045) and Meet Desai (25IT012) Full-Stack Web Dev Bootcamp registered; Kavya Mehta (25CS089) Robotics & Automation Exhibition cancelled – ordered by registered_at (06-join-query-result.png)

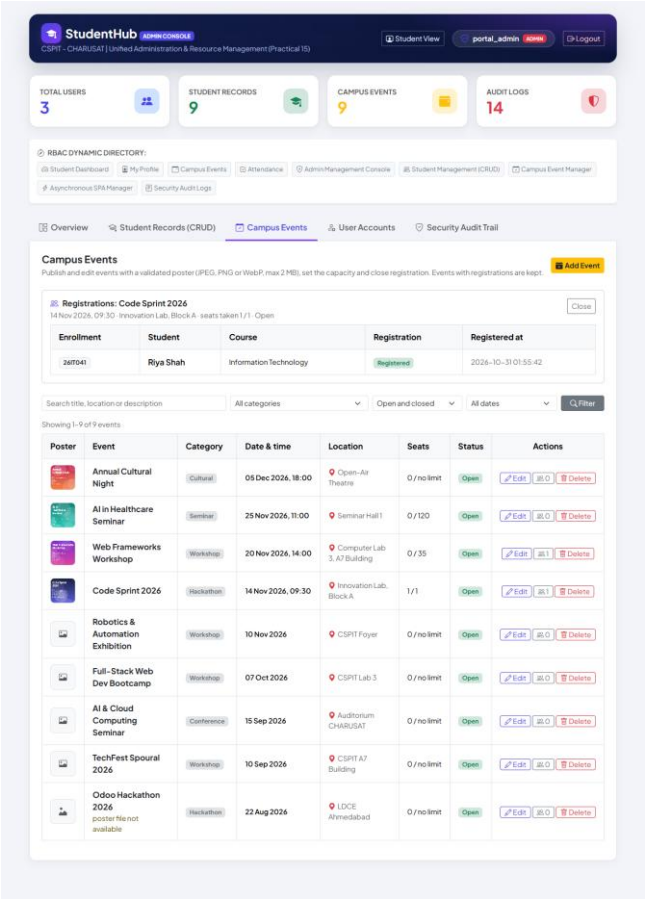


Figure 7: The registrations table in use (Practical 12, later step): Admin Console → Campus Events, registrations of Code Sprint 2026 (capacity 1, seats taken 1 / 1) – Riya Shah 26IT041, Registered; the Seats column shows seats taken / capacity and "no limit" where capacity is NULL (11-event-registrations.png)

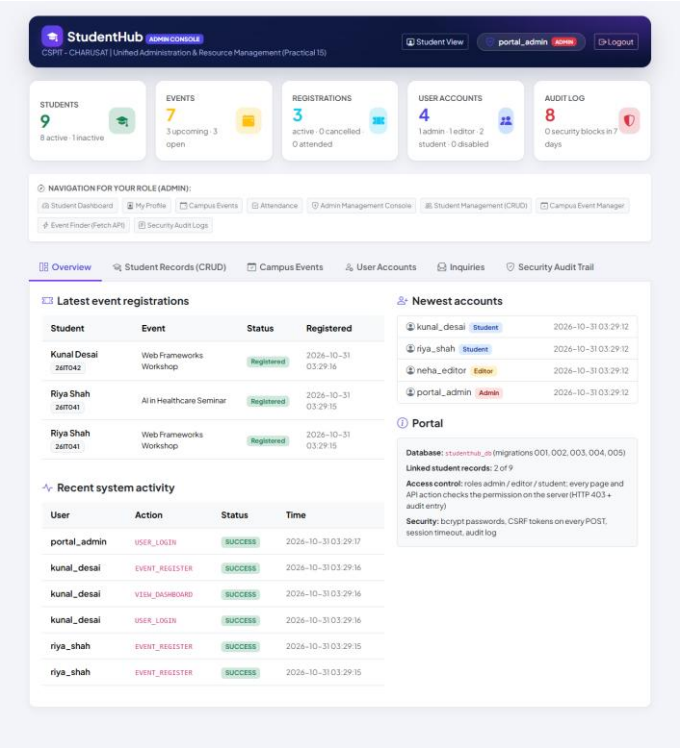


Figure 8: Admin dashboard (Practical 14, later step): Registrations counter and "Latest event registrations" from registrations JOIN students JOIN events; the Portal box names studenthub_db (migrations 001–004) and 2 of 9 student records linked to accounts (01-admin-dashboard.png)

10. Testing and Results

All tests were executed on 31 October 2026 with PHP 8.2.12 (XAMPP), MariaDB 10.4.32 and PHP's built-in server on Windows 11. The migration test creates the disposable databases `studenthub_mig001_test`, `studenthub_mig001_pre` and `studenthub_mig001_fresh` (each with a marker table) and drops them; the real `studenthub_db` is only read, by the `--verify-live` mode. The numbers are copied from the saved outputs.

Suite	Result	Output
`database/tests/test_001_practical8_foundation.py` (fixture, fictional data)	**88 / 88 passed**	`database/tests/test_001_output.txt`
… `--verify-live` – live database after the migration (read-only)	**24 / 24 passed**	`database/tests/test_001_verify_live_output.txt`
`tests/browser_database_evidence.py` – phpMyAdmin in Edge (Figures 1–6)	**6 / 6 passed**	`tests/browser_database_evidence_output.txt`
`tests/run_all.py` – row "Practical 8 – migration 001 and portal"	**88 / 0** (whole run 841 / 0)	`tests/run_all_output.txt`

`database/README.md` and TEST_REPORT §6–§8 record earlier runs with 87 checks (fixture 87 / 87, restored real pre-migration dump 87 / 87 – not committed because it contains real records); the saved output of the latest run

contains 88 checks, all passed.

Area (test_001_output.txt)	Checks	What was verified
Load + apply migration 001	1 + 23	every new column, key, FK and CHECK with exact type and rule; `001` recorded once; every original value unchanged (SHA-256, incl. password hashes); nothing invented
Linking students to accounts	3	only unique, verified matches with a student account (2 in the fixture); no admin/editor linked
Repeated run / rollback and re-apply	3 + 5	second run changes nothing; rollback restores the exact structure and data, runs twice; re-apply identical
Precondition stop	5	deleted-account audit entry or semester outside 1–8 → stop, nothing changed; demo file refused on an old database
Capacity rule / parallel requests	12 + 2	duplicates refused; capacity 2 → third student `event_full`; cancelled seat reused; NULL = no limit; **8 simultaneous processes for 3 seats → exactly 3**
CHECK constraints / foreign keys	4 + 12	capacity 0, semester 0 and 9 refused; missing references refused; CASCADE deletes; deleting an account keeps the record and audit entries (user_id → NULL)
Portal pages on the migrated copy	13	logins, Admin Console tabs and actions, student pages, 403 for a student, self-registration linked, no PHP warnings
Fresh install from studenthub_demo.sql	5	structure identical to an upgraded database; 001 and a re-import change nothing

Live check (24 / 24): after migration 001 was applied to the live `studenthub_db` (after a verified backup), `--verify-live` found all 19 structure checks satisfied, the structure identical to the migrated baseline, every original row and value unchanged (incl. password hashes), the links equal to the verified e-mail matches (5) and `registrations` empty at that time. **Later uses:** `tests/test_events.py` (94 / 94; 6 simultaneous registrations for 3 seats → exactly 3) and `tests/test_api.py` (72 / 72; 6 simultaneous API registrations → three 201 and three 409) exercise the same function (TEST_REPORT §9–§10).

11. Problems Found and Design Decisions

- **Audit log and stale sessions:** with the new foreign key on `audit_logs.user_id`, logging an action of an account deleted while its session was open would have failed the page; `studenthub/audit_logger.php` now catches MySQL error 1452 and stores the entry with `user_id = NULL` and the username kept (tested).
- **Key types:** a foreign key needs the exact referenced type, so the new keys are signed `INT(11)` like the portal's primary keys (Practical 8 used `INT UNSIGNED`); the preconditions check the types first.
- **No invented or silently changed data:** existing events get capacity and time NULL, legacy students without a semester stay valid, a semester outside 1–8 stops the migration, and ambiguous e-mails or admin/editor accounts are never linked.
- **Non-transactional DDL:** preconditions, idempotent steps, a verification block, a dump before migrating and a tested rollback replace the atomicity MariaDB cannot give. The fresh-install test was later updated to apply the newer migrations first (README).

12. Differences from the Original Practical and Limitations

Aspect	Original report (Practical-8/)	Integrated portal
Database	Own `studenthub_prac8`, recreated by `schema.sql`	Shared `studenthub_db`; design added by migration 001

Connection	PDO, native prepares, strict SQL mode, `config.local.php`	MySQLi, exception mode, environment variables, no fallback, HTTP 503; no strict mode
Business logic	Stored procedures `register_student_for_event`, `get_event_roster`	PHP transaction; rosters in the Admin Console (Practical 12)
Demo data	8 students, 5 events, 12 registrations	7 students, 5 events, no registrations or accounts
Evidence	`test_connection.php` page; 44 / 44 + 44 / 44 tests	phpMyAdmin on a disposable copy; 88 / 88, 24 / 24 live, 6 / 6 browser

- A plain `INSERT INTO registrations` bypasses the capacity rule – the portal must always use `register_student_for_event()`.
- Without strict SQL mode, values not covered by a CHECK (e.g. too-long text) are silently adjusted.
- Course names are free text. A self-registered student's e-mail is stored in `users` and `students`; editing the record in the Admin Console changes only `students.email`.
- Registrations cannot be marked `attended` from the interface (TEST_REPORT §9); only MariaDB 10.4.32 was tested.

13. Conclusion

The Practical 8 design – students and events stored once and linked by `registrations` with two CASCADE foreign keys and a unique student–event pair – is now part of the one database of the integrated portal. Migration 001 added it to the existing tables with read-only preconditions, a strict account-linking rule, CHECK constraints, a verification block and a tested rollback, without changing any existing value. The capacity rule runs as a PHP transaction that locks the event row, so eight simultaneous requests for three seats produced exactly three registrations. The portal connects with MySQLi in exception mode and never shows database details. 88 automated checks, 24 read-only live checks and 6 browser checks passed.

14. References

- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 – `database/`, `studenthub/db.php`, `studenthub/includes/event_registration.php`, `evidence/practical-08-integration/`, `TEST_REPORT.md`; commit `37b35e6`
- MariaDB Knowledge Base – CONSTRAINT (CHECK and FOREIGN KEY): https://mariadb.com/kb/en/constraint/
- MariaDB Knowledge Base – SELECT … FOR UPDATE: https://mariadb.com/kb/en/for-update/
- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- PHP Manual – mysqli_report: https://www.php.net/manual/en/function.mysqli-report.php
- phpMyAdmin documentation: https://docs.phpmyadmin.net/en/latest/
- OWASP – SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html